

RAPPORT D'ANALYSE

Choix du projet à consolider pour la suite de la SAÉ

Projet évalué : SoundStream

Type d'analyse : auto-évaluation, même grille que les audits externes (MusiQuali, HeyDo)

Équipe : Synapse Audio: K. Cassel, S.-E. Aljane, M.Hachim, N.Sefraoui , A. Sy

Client référent : M. Patrice Bringuier

Établissement : IUT de Villetaneuse Université Sorbonne Paris Nord BUT2 Informatique

Préambule et méthodologie

Dans le cadre de la reprise des projets du premier semestre, chaque équipe analyse plusieurs dépôts afin de décider sur quelle base poursuivre la SAÉ. Notre équipe a audité deux projets externes (MusiQuali, 56/100 ; HeyDo, 51/100 ; Rajouter les autres projets) puis appliqué exactement la même grille à notre propre projet, SoundStream. Ce rapport présente cette auto-évaluation.

Les cinq critères sont : (1) fonctionnalité et respect du besoin, (2) qualité du code, (3) gestion des données, (4) interface utilisateur, (5) installation et déploiement. Le besoin de référence est celui exprimé par M. Bringuier : système multi-site à gestion centralisée, supervision de l'état des lecteurs, synchronisation automatique des playlists, continuité de service en cas de coupure, consignment des diffusions et messages urgents prioritaires.

Critère 1: Fonctionnalité et respect du besoin

Ce qui fonctionne et constitue le cœur du projet

SoundStream traite réellement la dimension réseau, qui est le cœur de cette SAÉ. Le serveur Flask central gère une table de lecteurs (ajout, suppression, modification, identification par IP et organisation) et pousse effectivement les fichiers vers chaque lecteur distant via rsync et SSH (SongPlayerService, DevicesController). L'identifiant des lecteurs n'est pas figé en dur : on déclare autant de lecteurs que voulu, rattachés à leur organisation. Nous l'avons validé en conditions réelles sur deux lecteurs simultanés (synchronisation et lecture confirmées sur les deux).

- **Synchronisation automatique et planning** : l'ajout d'un fichier à une playlist déclenche la génération du fichier M3U et la synchronisation vers les lecteurs en ligne. Un ordonnanceur en arrière-plan identifie la playlist du créneau horaire courant et la pousse vers tous les lecteurs en ligne. Le planning hebdomadaire est éditable (assignation playlist / jour / heure de début).
- **Authentification et rôles** : connexion fonctionnelle avec trois rôles (admin, marketing, sales) et un décorateur reqrole qui protège les routes selon le rôle. Conforme au besoin.
- **Continuité et reprise au démarrage** : chaque lecteur conserve ses fichiers et sa playlist en local ; nous avons mis en place le redémarrage automatique du lecteur audio

au boot (service utilisateur systemd + lingering), ce qui répond à l'exigence « ça redémarre tout seul » après une coupure.

Fonctionnalités encore partielles ou à finaliser

- **Publicités programmées** : l'upload d'un message publicitaire fonctionne et est consigné, mais la diffusion à horaire programmé (heure/date du formulaire) n'est pas encore reliée à l'ordonnanceur le déclenchement récurrent reste à câbler.
- **Now Playing et reprise au timestamp exact** : la visualisation « en cours de lecture » par lecteur (US-029) et la reprise au morceau/timestamp précis après coupure (US-013) ne sont pas encore implémentées ; l'architecture les permet déjà.
- **Alertes** : une alerte e-mail est envoyée quand un lecteur passe hors ligne, mais le système d'alertes visuelles dans l'interface (badge, playlist obsolète > 7 jours) reste à ajouter.

Note critère 1 : 16/20

Pistes : câbler la diffusion publicitaire à l'ordonnanceur ; ajouter la page *Now Playing* (lecture de *mpc current*) ; finaliser la sauvegarde périodique de position pour la reprise exacte ; transformer les anomalies en alertes visibles dans l'interface.

Critère 2: Qualité du code

L'architecture est un véritable atout : séparation nette Controllers / Services / DAO / Models, avec des interfaces de DAO (UserDAOInterface, SongPlayerDAOInterface, etc.) qui définissent un contrat clair. Pour un BUT2, cette intention MVC en couches, appliquée de manière cohérente sur l'ensemble des entités, est bien au-dessus du minimum attendu et facilite la maintenance et les tests futurs.

- **Dépôt globalement assaini** : contrairement à d'autres projets analysés, le dépôt ne contient pas de fichiers de gabarit étrangers ni de marqueurs de conflit Git commités. La création du schéma est centralisée dans un seul fichier (schema.sql) avec un initdb dédié, ce qui évite les « sources de vérité multiples » constatées ailleurs.
- **Gestion Git en cours de professionnalisation** : nous travaillons désormais par branches dédiées et pull requests ciblées sur le bon dépôt, avec relecture par un pair avant fusion. C'est un progrès net par rapport à un historique désordonné.

Faiblesses que nous reconnaissons

- **Code mort et méthodes à fiabiliser** : certaines méthodes héritées ne sont plus utilisées ou comportent des défauts (une méthode de programmation appelle une requête inexistante, une autre lit une clé absente du résultat SQL). Elles doivent être corrigées ou supprimées.
- **Opérations sensibles en GET** : plusieurs suppressions (utilisateur, lecteur) sont accessibles en GET ; elles doivent passer en POST.
- **Mode debug et absence de tests** : le serveur tourne en debug ; surtout, le dossier de tests est vide alors que le critère mentionne un code « testé ». Une suite de tests (authentification, rôles, intégrité, ping simulé) est une priorité et relève directement de notre poste qualité.

Note critère 2 : 13/20

Pistes : supprimer le code mort et les méthodes buguées ; nettoyer les commentaires ; passer les actions destructrices en POST + CSRF ; désactiver debug en déploiement ; construire une vraie suite de tests.

Critère 3: Gestion des données

Deux bonnes pratiques structurantes méritent d'être saluées : les mots de passe sont hachés avec bcrypt, et l'ensemble des requêtes utilisent des paramètres liés (« ? »), ce qui protège des injections SQL. C'est rigoureux et conforme aux attentes.

→ **Modèle de données normalisé** : c'est un avantage net. La relation playlist–fichier passe par une vraie table d'association (composition), le planning par une table planned référençant les playlists, et le lien utilisateur–organisation par work_link. Nous évitons donc l'anti-patron des relations stockées en chaînes de caractères (« 1|2|3 ») et la recopie de données dans le planning, qui pénalisaient d'autres projets. Le schéma est unique et cohérent (schema.sql).

Faiblesses que nous reconnaissons

→ **Base dans un dossier servi publiquement** : la base est stockée sous static/database/, donc théoriquement téléchargeable via une URL /static/. Même avec des mots de passe hachés, exposer la base reste une faute de fond : c'est le même défaut que celui relevé dans les projets audités, et nous l'assumons. Le déplacer hors de static/ est en tête de notre plan de consolidation.

→ **Secrets et comptes par défaut** : une clé secrète Flask et des identifiants d'envoi d'e-mail sont écrits en dur, et les comptes de départ partagent un mot de passe par défaut. À externaliser en variables d'environnement et à forcer au premier changement.

→ **Intégrité référentielle** : les clés étrangères sont déclarées mais SQLite ne les applique pas sans activation explicite (PRAGMA foreign_keys). À activer pour garantir les suppressions en cascade.

Note critère 3 : 13/20

Pistes : déplacer la base hors de static/ ; externaliser les secrets en variables d'environnement et supprimer les comptes/mots de passe par défaut ; activer PRAGMA foreign_keys sur chaque connexion.

Critère 4: Interface utilisateur

Pour une application dont la mission est la supervision, l'interface répond aux attentes là où d'autres projets échouent.

→ **Supervision en temps réel** : le tableau des lecteurs se rafraîchit automatiquement (interrogation périodique du serveur) et affiche l'état ONLINE/OFFLINE avec un code couleur (vert/rouge) ainsi que les compteurs en ligne/hors ligne. C'est précisément le « temps réel » qui manquait aux interfaces des projets audités, où une panne n'apparaissait qu'après rechargement manuel.

→ **Actions réellement câblées** : synchronisation groupée (« Sync All ») avec barre de progression, export CSV des journaux, navigation adaptée au rôle (l'admin, le marketing et le sales ne voient pas les mêmes menus). Les boutons déclenchent de vraies routes, pas des cadres décoratifs.

- **Confort** : horloge en direct, formulaires de gestion (utilisateurs, lecteurs, playlists) complètement fonctionnels, page d'aide en cas de mot de passe oublié.

Faiblesses que nous reconnaissons

- **Alerte d'urgence non annulable depuis l'interface** : il manque un bouton d'arrêt immédiat de l'annonce urgente côté interface.
- **Mise en page perfectible** : de nombreux styles sont écrits en ligne dans les gabarits plutôt que centralisés, et l'interface vise le poste de travail (desktop), conformément au cahier des charges, mais reste peu adaptée au mobile.

Note critère 4 : 14/20

Pistes : ajouter un bouton d'arrêt de l'annonce urgente ; afficher les alertes (badge) directement dans le tableau ; centraliser les styles ; harmoniser les pages.

Critère 5: Installation et déploiement

SoundStream est réellement orienté Linux (rsync, SSH, MPD, scripts shell, environnement virtuel Python), ce qui respecte l'exigence technique du cahier des charges contrairement à des projets fournissant un script de démarrage Windows.

- **Documentation et dépendances** : le dépôt fournit un README expliquant le lancement (environnement virtuel, installation des dépendances, démarrage) et un fichier requirements.txt avec les versions figées deux éléments absents de plusieurs projets audités.
- **Déploiement prouvé** : la base se crée automatiquement au lancement via initdb, et nous avons déroulé de bout en bout l'installation d'un lecteur (machine Linux, configuration réseau, SSH par clé, MPD, démarrage automatique), validée sur deux lecteurs.

Faiblesses que nous reconnaissons

- **Étapes manuelles persistantes** : la mise en service d'un lecteur suppose encore quelques étapes manuelles (échange de clé SSH, configuration MPD). Un script de provisionnement unique rendrait le « branche et ça marche » plus proche.
- **Configuration en dur** : l'adresse et le port du serveur figurent en dur à quelques endroits ; ils gagneraient à être externalisés dans une configuration.

Note critère 5 : 14/20

Pistes : fournir un script de provisionnement lecteur unique ; externaliser hôte/port dans une configuration ; documenter le scénario multi-lecteurs.

Conclusion et décision

Critère	Note
1 — Fonctionnalité / respect du besoin	16 / 20
2 — Qualité du code	13 / 20
3 — Gestion des données	13 / 20
4 — Interface utilisateur	14 / 20
5 — Installation & déploiement	14 / 20
TOTAL	70 / 100

Avec un total de 70/100, SoundStream se place devant les deux projets que nous avons audités (MusiQuali, 56/100 ; HeyDo, 51/100). Cet écart ne tient pas à un parti pris : il reflète le fait que SoundStream couvre déjà le cœur fonctionnel demandé par le client, là où les autres projets présentaient des manques structurants.

Trois éléments décisifs justifient notre choix de manière objective. D'abord, la communication réseau multi-site est réelle et prouvée sur deux lecteurs simultanés. Ensuite, l'interruption par message urgent fonctionnalité centrale absente du moteur de diffusion de MusiQuali et totalement absente chez HeyDo est implémentée chez nous. Enfin, la consignation centrale des diffusions et la supervision UP/KO avec état persisté et rafraîchissement automatique fonctionnent, alors qu'elles restaient au stade d'actions manuelles ou non opérationnelles ailleurs.

À ces forces s'ajoutent un modèle de données normalisé et un dépôt assaini, deux points sur lesquels les projets audités perdaient le plus de terrain. Nous assumons en contrepartie nos propres faiblesses : base à sortir du dossier public, secrets à externaliser, tests à écrire, code mort à nettoyer et les avons inscrites dans une feuille de route claire. Les identifier nous-mêmes est précisément ce qui distingue un projet maîtrisé.

Reprendre MusiQuali ou HeyDo nous obligerait à reconstruire le moteur de diffusion (canaux, messages urgents), à mettre en place la remontée et l'affichage des journaux et à refondre le modèle de données un volume de travail supérieur à la consolidation de SoundStream, déjà aligné sur ces exigences et dont nous maîtrisons l'architecture.

Décision finale : notre équipe décide de poursuivre la SAÉ sur la base de son propre projet, SoundStream, et d'orienter l'effort restant vers la consolidation décrite dans les pistes ci-dessus.